AI VIET NAM
@aivietnam.edu.vn

Module 02 – Extra Class

Vector - Matrix

Nguyen Quoc Thai

Objectives

Vector Operations

- Addition
- Subtraction
- Multiplication
- Division
- Dot Product

Cosine Similarity

- Compute Cosine Similarity
- Image Similarity

Matrix Operations

- Addition
- Subtraction
- Multiplication
- Division
- Dot Product
- Transpose
- Determinant
- Inverse

Outline

SECTION 1

Vector Operations

SECTION 2

Matrix Operations

SECTION 3

Cosine Similarity

Vector Operations

Vector

$$\vec{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} \in \begin{bmatrix} \mathcal{R} \\ \mathcal{R} \\ \mathcal{R} \end{bmatrix} = \mathcal{R}^3$$

n is a natural number

$\mathcal{R}$ is a set of real numbers

$\vec{v}$ has a length of n and contain real numbers

$\vec{v} \in \mathcal{R}^n$

```python
import numpy as np

list1 = [1, 2, 3]
vector1 = np.array(list1)
print(vector1)

vector2 = np.arange(1, 4, 2)
print(vector2)
```

✓ 0.0s

```
[1 2 3]
[1 3]
```

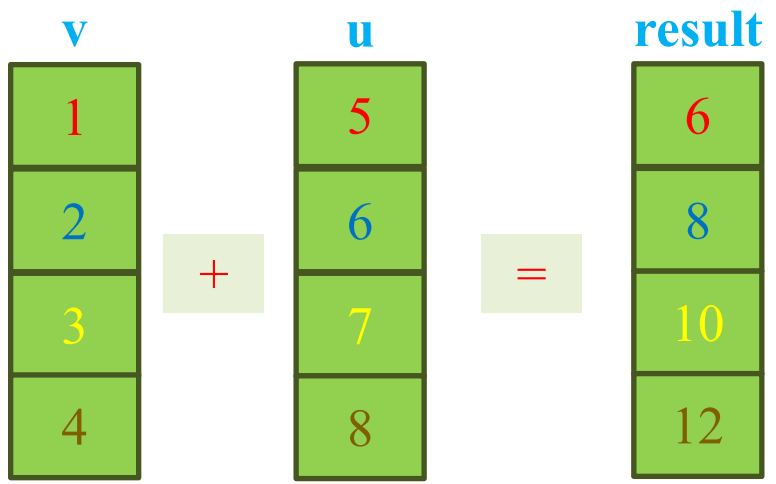
Vector Operations

Vector Addition

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} + \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} + \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} v_1 + u_1 \\ \dots \\ v_n + u_n \end{bmatrix}$$

$$\vec{u} + \vec{v} = \vec{v} + \vec{u}$$



Vector Operations

Vector Addition

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} + \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} + \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} v_1 + u_1 \\ \dots \\ v_n + u_n \end{bmatrix}$$

$$\vec{v} + \vec{v} = \vec{v} + \vec{v}$$

```python
import numpy as np

u = np.array([1, 2, 3, 4])
print(u)

v = np.array([5, 6, 7, 8])
print(v)

print(u+v)
print(np.add(u,v))
```

✓ 0.0s

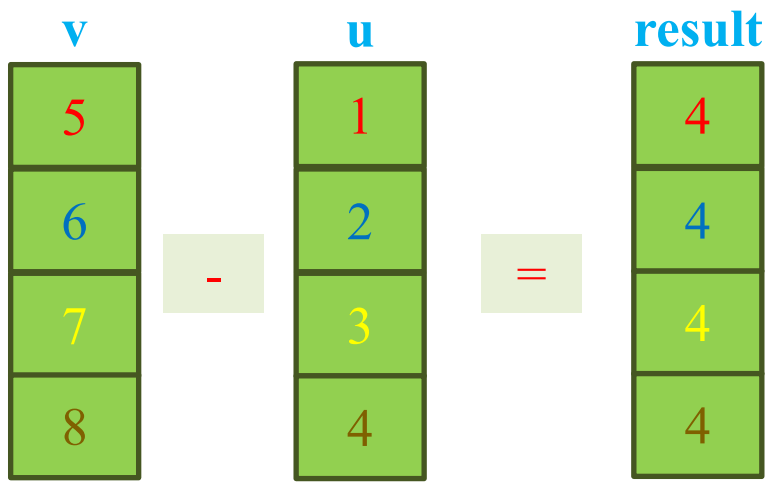
```
[1 2 3 4]
[5 6 7 8]
[ 6  8 10 12]
[ 6  8 10 12]
```

Vector Operations

Vector Subtraction

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} - \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} - \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} v_1 - u_1 \\ \dots \\ v_n - u_n \end{bmatrix}$$



Vector Operations

Vector Subtraction

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} - \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} - \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} v_1 - u_1 \\ \dots \\ v_n - u_n \end{bmatrix}$$

```python
import numpy as np

u = np.array([1, 2, 3, 4])
print(u)

v = np.array([5, 6, 7, 8])
print(v)

print(v-u)
print(np.subtract(v,u))
```

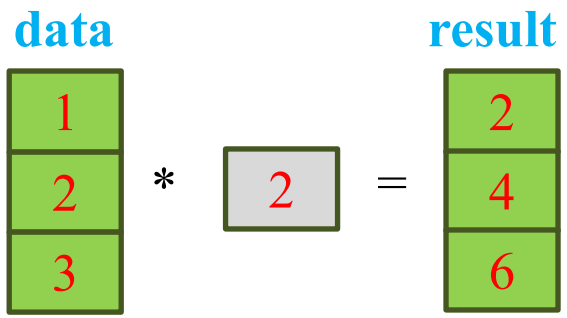
✓ 0.0s

```
[1 2 3 4]
[5 6 7 8]
[4 4 4 4]
[4 4 4 4]
```

Vector Operations

Multiplication of Vectors with Scalar

$$\alpha\vec{u} = \alpha \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} \alpha u_1 \\ \dots \\ \alpha u_n \end{bmatrix}$$




```python
import numpy as np

u = np.array([1, 2, 3])
alpha = 2

result = alpha*u
print(result)
```

✓ 0.0s

```
[2 4 6]
```

Vector Operations

Length of a Vector

$$\vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\|\vec{u}\| = \sqrt{u_1^2 + \cdots + u_n^2}$$

```python
import numpy as np

u = np.array([1, 2, 4, 2])
length = np.linalg.norm(u)
print(length)
```

✓ 0.0s

```
5.0
```

1
2
4
2

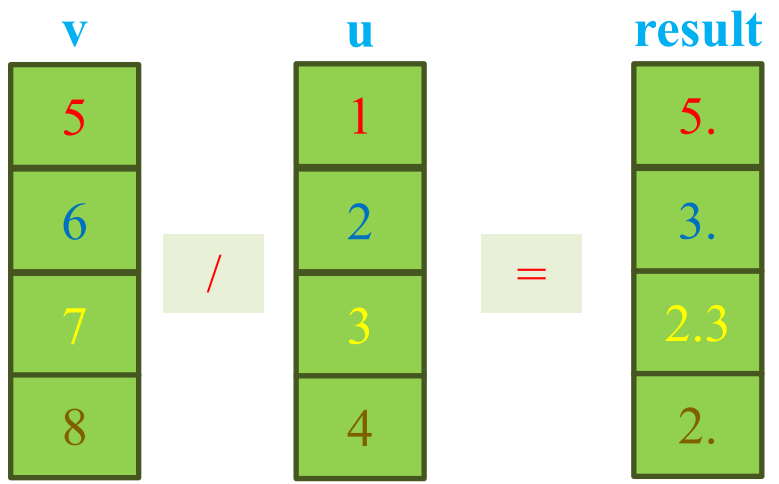
$$\|\vec{u}\| = \sqrt{1^2 + 2^2 + 4^2 + 2^2}$$

Vector Operations

Vector Division

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} + \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} / \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} v_1/u_1 \\ \dots \\ v_n/u_n \end{bmatrix}$$



Vector Operations

Vector Division

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} + \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} / \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} v_1/u_1 \\ \dots \\ v_n/u_n \end{bmatrix}$$

```python
import numpy as np

u = np.array([1, 2, 3, 4])
print(u)

v = np.array([5, 6, 7, 8])
print(v)

print(v/u)
print(np.divide(v,u))
print(v//u)
print(v%u)
```

✓ 0.0s

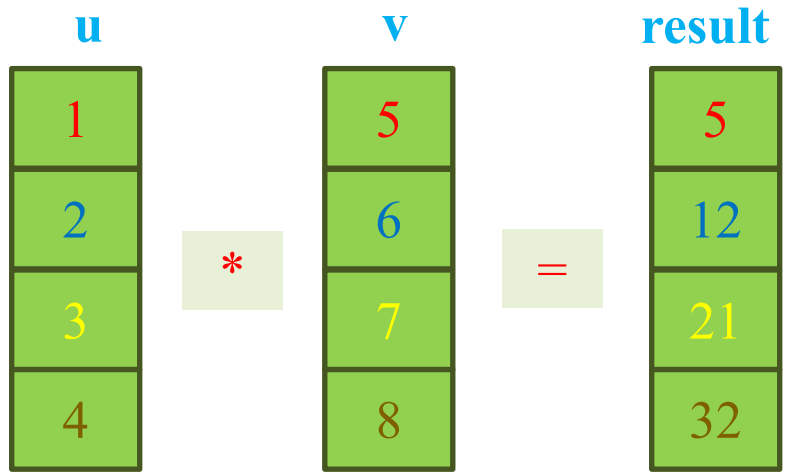
```
[1 2 3 4]
[5 6 7 8]
[5.         3.         2.33333333 2.        ]
[5.         3.         2.33333333 2.        ]
[5 3 2 2]
[0 0 1 0]
```

Vector Operations

Hadamard Product

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} \odot \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \odot \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} v_1 \times u_1 \\ \dots \\ v_n \times u_n \end{bmatrix}$$



Vector Operations

Hadamard Product

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} \odot \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \odot \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} v_1 \times u_1 \\ \dots \\ v_n \times u_n \end{bmatrix}$$

```python
import numpy as np

u = np.array([1, 2, 3, 4])
print(u)

v = np.array([5, 6, 7, 8])
print(v)

print(v*u)
print(u*v)
print(np.multiply(u, v))
```

✓ 0.0s

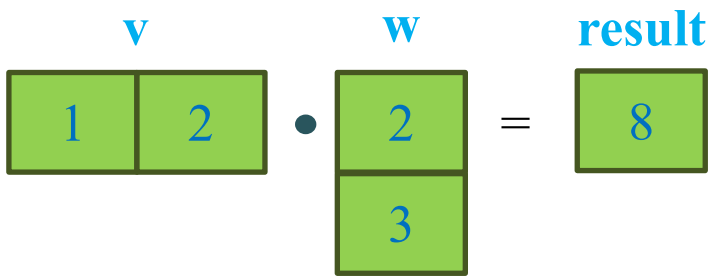
```
[1 2 3 4]
[5 6 7 8]
[ 5 12 21 32]
[ 5 12 21 32]
[ 5 12 21 32]
```

Vector Operations

Dot Product

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} \cdot \vec{u} = v_1 \times u_1 + \cdots + v_n \times u_n$$




```python
import numpy as np

u = np.array([1, 2])
print(u)

v = np.array([2, 3])
print(v)

print(u.dot(v))
print(np.dot(u, v))
print(np.dot(v, u))
```

✓ 0.0s

```
[1 2]
[2 3]
8
8
8
```

Outline

SECTION 1

Vector Operations

SECTION 3

Cosine Similarity

SECTION 2

Matrix Operations

Matrix Operations

Matrix

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \in \begin{bmatrix} \mathcal{R} & \mathcal{R} \\ \mathcal{R} & \mathcal{R} \\ \mathcal{R} & \mathcal{R} \end{bmatrix} = \mathcal{R}^{3\times 2}$$

Matrix A has the shape of rectangle

Has m rows and n columns

Use capital letter

$A \in \mathcal{R}^{m\times n}$

```python
import numpy as np

matrix1 = np.array([[1, 2], [3, 4]])
print(matrix1)

matrix2 = np.random.rand(2, 3)
print(matrix2)
```

✓ 0.0s

```
[[1 2]
 [3 4]]
[[0.72853495 0.66156034 0.71303952]
 [0.90718281 0.30636326 0.21290792]]
```

Matrix Operations

Matrix Addition

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & \dots & b_{1n} \\ \dots & \dots & \dots \\ b_{m1} & \dots & b_{mn} \end{bmatrix}$$

$$A + B = \begin{bmatrix} (a_{11} + b_{11}) & \dots & (a_{1n} + b_{1n}) \\ \dots & \dots & \dots \\ (a_{m1} + b_{m1}) & \dots & (a_{mn} + b_{mn}) \end{bmatrix}$$

A			B			C	
1	2	+	3	4	=	4	6
3	4		4	5		7	9

```python
import numpy as np

A = np.array([[1, 2], [3, 4]])
print(A)
B = np.array([[3, 4], [4, 5]])
print(B)

print(A+B)
print(np.add(A, B))
```

✓ 0.0s

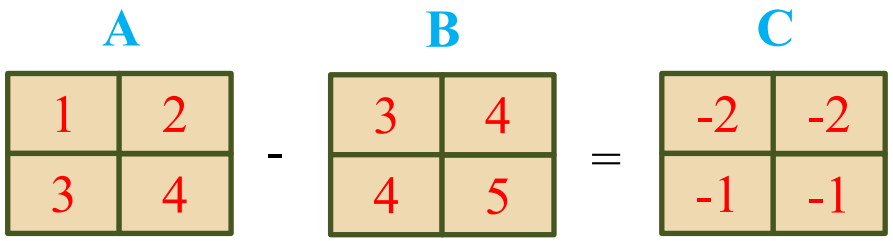
```
[[1 2]
 [3 4]]
[[3 4]
 [4 5]]
[[4 6]
 [7 9]]
[[4 6]
 [7 9]]
```

Matrix Operations

Matrix Subtraction

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & \dots & b_{1n} \\ \dots & \dots & \dots \\ b_{m1} & \dots & b_{mn} \end{bmatrix}$$

$$A - B = \begin{bmatrix} (a_{11} - b_{11}) & \dots & (a_{1n} - b_{1n}) \\ \dots & \dots & \dots \\ (a_{m1} - b_{m1}) & \dots & (a_{mn} - b_{mn}) \end{bmatrix}$$




```
1  A = np.array([[1, 2], [3, 4]])
2  print(A)
3  B = np.array([[3, 4], [4, 5]])
4  print(B)
5
6  print(A-B)
7  print(np.subtract(A, B))
```

✓ 0.0s

```
[[1 2]
 [3 4]]
[[3 4]
 [4 5]]
[[-2 -2]
 [-1 -1]]
[[-2 -2]
 [-1 -1]]
```

Matrix Operations

Matrix Subtraction

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & \dots & b_{1n} \\ \dots & \dots & \dots \\ b_{m1} & \dots & b_{mn} \end{bmatrix}$$

$$A/B = \begin{bmatrix} (a_{11}/b_{11}) & \dots & (a_{1n}/b_{1n}) \\ \dots & \dots & \dots \\ (a_{m1}/b_{m1}) & \dots & (a_{mn}/b_{mn}) \end{bmatrix}$$

A			B			C	
1	2	/	3	4	=	0.33	0.5
3	4		4	5		0.75	0.8

```
1  A = np.array([[1, 2], [3, 4]])
2  print(A)
3  B = np.array([[3, 4], [4, 5]])
4  print(B)
5
6  print(A/B)
7  print(np.divide(A, B))
```

✓ 0.0s

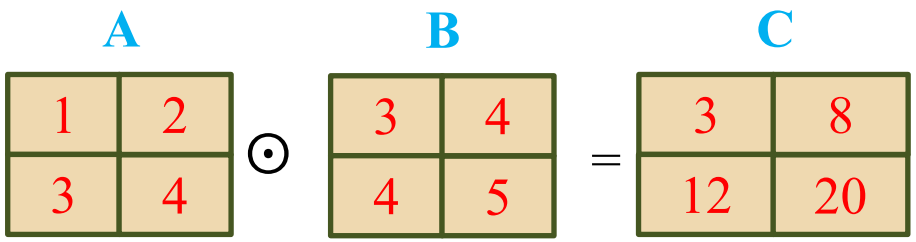
```
[[1 2]
 [3 4]]
[[3 4]
 [4 5]]
[[0.33333333 0.5       ]
 [0.75       0.8       ]]
[[0.33333333 0.5       ]
 [0.75       0.8       ]]
```

Matrix Operations

Hadamard Product

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & \dots & b_{1n} \\ \dots & \dots & \dots \\ b_{m1} & \dots & b_{mn} \end{bmatrix}$$

$$A \odot B = \begin{bmatrix} (a_{11} * b_{11}) & \dots & (a_{1n} * b_{1n}) \\ \dots & \dots & \dots \\ (a_{m1} * b_{m1}) & \dots & (a_{mn} * b_{mn}) \end{bmatrix}$$




```
1  A = np.array([[1, 2], [3, 4]])
2  print(A)
3  B = np.array([[3, 4], [4, 5]])
4  print(B)
5
6  print(A*B)
7  print(np.multiply(A, B))
```

✓ 0.0s

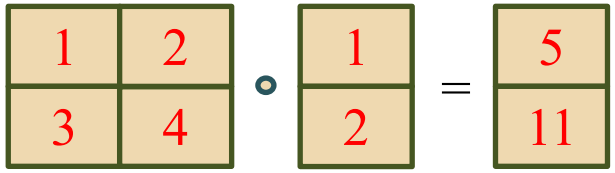
```
[[1 2]
 [3 4]]
[[3 4]
 [4 5]]
[[ 3  8]
 [12 20]]
[[ 3  8]
 [12 20]]
```

Matrix Operations

Dot Product / Matrix-vector Multiplication

$$\mathrm{A} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \cdots & \cdots & \cdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \quad \vec{\boldsymbol{x}} = \begin{bmatrix} x_1 \\ \cdots \\ x_n \end{bmatrix} \quad \mathrm{A} \in \mathcal{R}^{m \times n}$$

$$\mathrm{C} = \mathrm{A}\vec{\boldsymbol{x}} \quad \text{where } c_i = \sum_{l=1}^{n} a_{il} x_l$$




```
1  A = np.array([[1, 2], [3, 4]])
2  print(A)
3  x = np.array([1, 2])
4  print(x)
5
6  print(A.dot(x))
7  print(x.dot(A))
```

✓ 0.0s

```
[[1 2]
 [3 4]]
[1 2]
[ 5 11]
[ 7 10]
```

Matrix Operations

Dot Product / Matrix-matrix Multiplication

$$\mathrm{A} = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad \mathrm{B} = \begin{bmatrix} b_{11} & \dots & b_{1k} \\ \dots & \dots & \dots \\ b_{n1} & \dots & b_{nk} \end{bmatrix}$$

$$\mathrm{A} \in \mathcal{R}^{m \times n} \qquad \mathrm{B} \in \mathcal{R}^{n \times k}$$

$$\mathrm{C} = \mathrm{AB} \qquad c_{ij} = \sum_{l=1}^{n} a_{il} b_{lj}$$

$$\mathrm{AB} = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} b_{11} & \dots & b_{1k} \\ \dots & \dots & \dots \\ b_{n1} & \dots & b_{nk} \end{bmatrix} = \begin{bmatrix} (a_{11}b_{11} + \cdots + a_{1n}b_{n1}) & \dots & (a_{1n}b_{1k} + \cdots + a_{1n}b_{nk}) \\ \dots & \dots & \dots \\ (a_{m1}b_{11} + \cdots + a_{mn}b_{n1}) & \dots & (a_{m1}b_{1k} + \cdots + a_{mn}b_{nk}) \end{bmatrix}$$

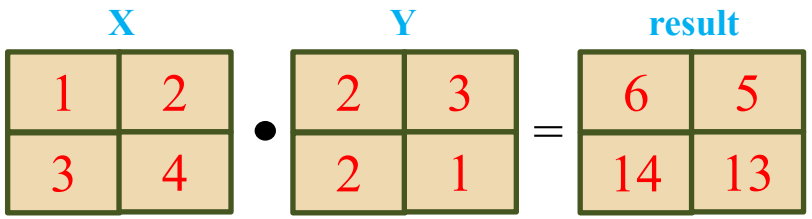
Matrix Operations

Dot Product / Matrix-matrix Multiplication

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & \dots & b_{1k} \\ \dots & \dots & \dots \\ b_{n1} & \dots & b_{nk} \end{bmatrix}$$

$$A \in \mathcal{R}^{m \times n} \qquad B \in \mathcal{R}^{n \times k}$$

$$C = AB \qquad c_{ij} = \sum_{l=1}^{n} a_{il} b_{lj}$$




```
1  A = np.array([[1, 2], [3, 4]])
2  print(A)
3  B = np.array([[2, 3], [2, 1]])
4  print(B)
5
6  print(A.dot(B))
7  print(np.dot(A, B))
8  print(B.dot(A))
```

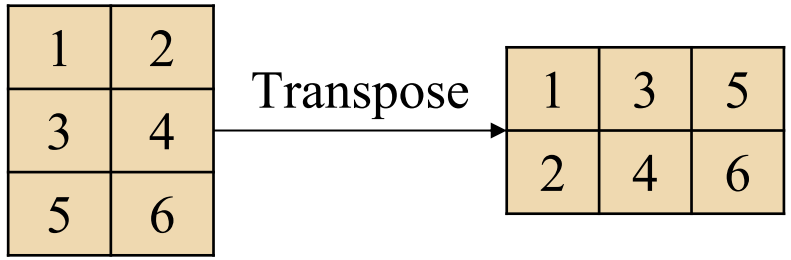
✓ 0.0s

```
[[1 2]
 [3 4]]
[[2 3]
 [2 1]]
[[ 6  5]
 [14 13]]
[[ 6  5]
 [14 13]]
[[11 16]
 [ 5  8]]
```

Matrix Operations

Transpose

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \cdots & \cdots & \cdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \quad A^T = \begin{bmatrix} a_{11} & \cdots & a_{m1} \\ \cdots & \cdots & \cdots \\ a_{1n} & \cdots & a_{mn} \end{bmatrix}$$




```
1  A = np.array([
2      [1, 2], [3, 4], [5, 6]
3  ])
4  print(A)
5
6  print(A.T)
```

✓ 0.0s

```
[[1 2]
 [3 4]
 [5 6]]
[[1 3 5]
 [2 4 6]]
```

Matrix Operations

Determinant

The ***determinant*** of a 2×2 matrix is the number

$$\det\begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

```
1  A = np.array([
2      [1, 2], [3, 4]
3  ])
4  print(A)
5
6  print(np.linalg.det(A))
```

✓ 0.0s

```
[[1 2]
 [3 4]]
-2.0000000000000004
```

Matrix Operations

Inverse

Definition. Let A be an $n \times n$ (square) matrix.

$$AB = I \quad and \quad BA = I$$

$B(n \times n)$: ***inverse*** of $A, B = A^{-1}$

$$Let\ A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

1.if $\det(A) \neq 0$, A: *invertible*

$$\mathrm{B} = A^{-1} = \frac{1}{\det(A)} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

2. if $\det(A) = 0$, *then A is not invertible*

Let

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}$$

$$\det(A) = 1 \cdot 1 - 2 \cdot 3 = -5,$$

$$\mathrm{B} = \begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}^{-1} = \frac{1}{\det(\mathrm{A})} \begin{pmatrix} 1 & -2 \\ -3 & 1 \end{pmatrix}$$

$$= -\frac{1}{5} \begin{pmatrix} 1 & -2 \\ -3 & 1 \end{pmatrix} = \begin{pmatrix} -1/5 & 2/5 \\ 3/5 & -1/5 \end{pmatrix}$$

Matrix Operations

Inverse

Definition. Let A be an $n \times n$ (square) matrix.

$$AB = I \quad and \quad BA = I$$

$$B(n \times n): \textbf{\textit{inverse}} \text{ of } A, B = A^{-1}$$

$$Let\ A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

$$1.if\ \det(A) \neq 0, A:\ invertible$$

$$\mathrm{B} = A^{-1} = \frac{1}{\det(A)} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

$$2.\ if\ \det(A) = 0, then\ A\ is\ not\ invertible$$

```
1  A = np.array([
2      [1, 2], [4, 6]
3  ])
4  print(A)
5
6  print(np.linalg.det(A))
7  B = np.linalg.inv(A)
8  print(B)
```

✓ 0.0s

```
[[1 2]
 [4 6]]
-2.0
[[-3.   1. ]
 [ 2.  -0.5]]
```

```
1  print(A.dot(B))
2  print(B.dot(A))
```

✓ 0.0s

```
[[1. 0.]
 [0. 1.]]
[[1. 0.]
 [0. 1.]]
```


QUIZ TIME

Outline

SECTION 1

Vector Operations

SECTION 3

Cosine Similarity

SECTION 2

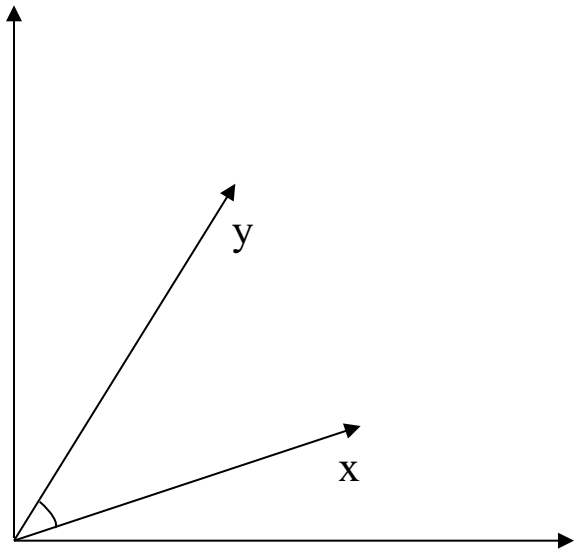
Matrix Operations

Cosine Similarity

Cosine Similarity

- Measure the similarity between two vectors

$$\mathbf{cs}(\vec{\mathbf{x}}, \vec{\mathbf{y}}) = \frac{\vec{x}.\vec{y}}{\|\vec{x}\|\|\vec{y}\|} = \frac{\sum_1^n x_i y_i}{\sqrt{\sum_1^n x_i^2}\sqrt{\sum_1^n y_i^2}}$$



Cosine Similarity

Cosine Similarity

- Measure the similarity between two vectors

$$\mathbf{cs}(\vec{\mathbf{x}}, \vec{\mathbf{y}}) = \frac{\vec{x}.\vec{y}}{\|\vec{x}\|\|\vec{y}\|} = \frac{\sum_1^n x_i y_i}{\sqrt{\sum_1^n x_i^2}\sqrt{\sum_1^n y_i^2}}$$

$$\vec{\mathrm{x}} = [4, 2, 1, 2]^T$$
$$\vec{\mathrm{y}} = [1, 2, 2, 0]^T$$

$$\mathrm{cs}(\vec{\mathrm{x}}, \vec{\mathrm{y}}) = \frac{4 * 1 + 2 * 2 + 1 * 2 + 2 * 0}{\sqrt{4^2 + 2^2 + 1^2 + 2^2}\sqrt{1^2 + 2^2 + 2^2 + 0}} = \frac{10}{\sqrt{25}\sqrt{9}} = \frac{10}{15} = 0.67$$

Cosine Similarity

Cosine Similarity

- Measure the similarity between two vectors

$$\mathbf{cs}(\vec{\mathbf{x}}, \vec{\mathbf{y}}) = \frac{\vec{x}.\vec{y}}{\|\vec{x}\|\|\vec{y}\|} = \frac{\sum_1^n x_i y_i}{\sqrt{\sum_1^n x_i^2}\sqrt{\sum_1^n y_i^2}}$$

```python
import numpy as np

A = np.array([1, 2, 3])
B = np.array([4, 5, 6])

print("A:", A)
print("B:", B)

# compute cosine similarity
cosine = np.dot(A,B)/(np.linalg.norm(A)*np.linalg.norm(B))
print("Cosine Similarity:", cosine)
```

✓ 0.0s

```
A: [1 2 3]
B: [4 5 6]
Cosine Similarity: 0.9746318461970762
```

Cosine Similarity

Image Similarity

- Measure the similarity between two images

Image 01

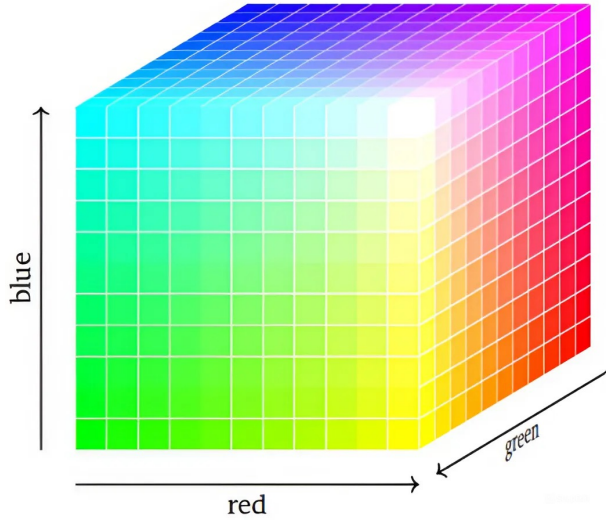
Image 02

Query image

Cosine Similarity

Image Similarity

- Measure the similarity between two image

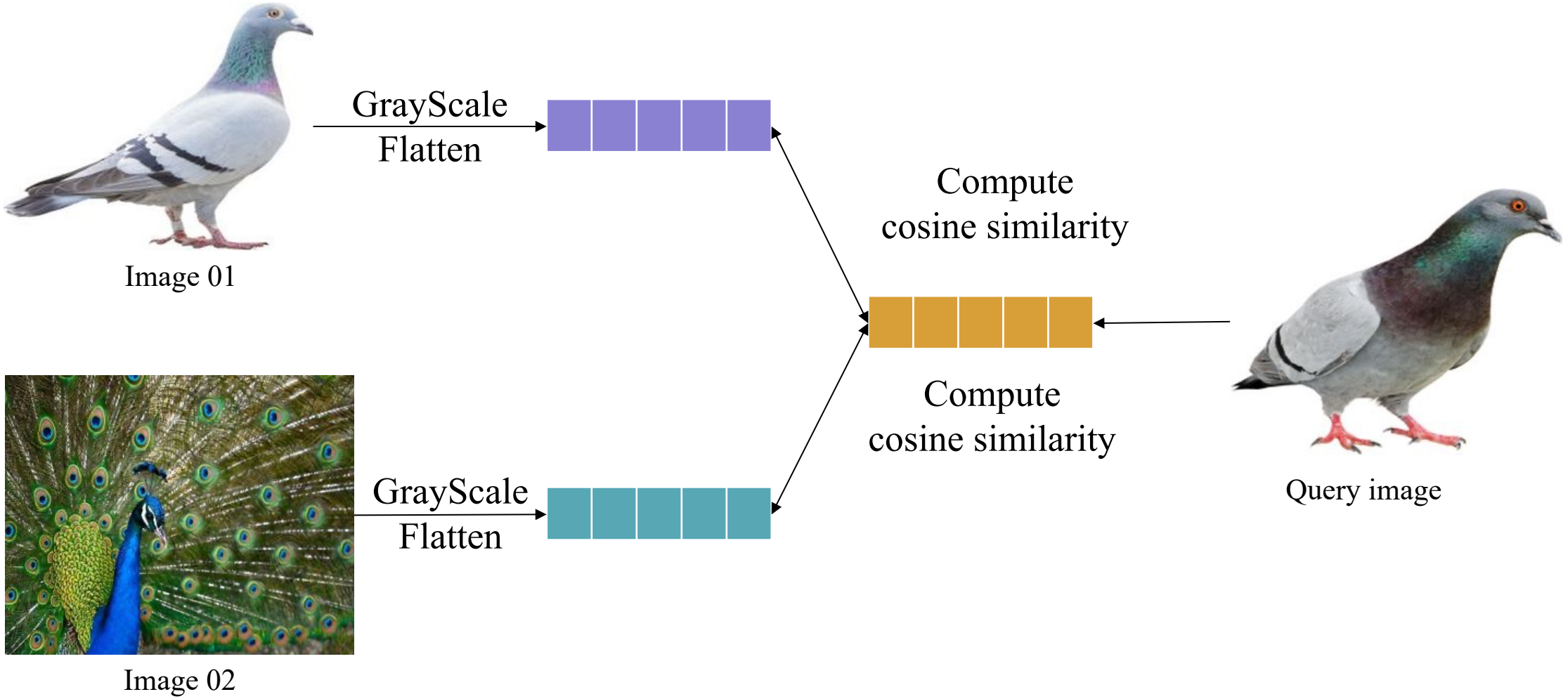


$$\text{Pixel p} = \begin{bmatrix} r \\ g \\ b \end{bmatrix}$$

$$0 \leq r, g, b \leq 255$$

Cosine Similarity

Image Similarity



Cosine Similarity

Image Similarity

- Load image

```
1  from PIL import Image
2
3  image1 = Image.open('./data/image1.jpg')
4  image2 = Image.open('./data/image2.jpg')
5  query_image = Image.open('./data/query_image.jpg')
```

```
1  image1_data = np.array(image1)
2  image2_data = np.array(image2)
3  query_image_data = np.array(query_image)
4  image1_data.shape, image2_data.shape, query_image_data.shape
```

Cosine Similarity

Image Similarity

- Convert 2D array, Flatten

```
image1_data = np.sum(image1_data, axis=-1, keepdims=1)/3
image2_data = np.sum(image2_data, axis=-1, keepdims=1)/3
query_image_data = np.sum(query_image_data, axis=-1, keepdims=1)/3
```

```
image1_flatten = image1_data.flatten()
image2_flatten = image2_data.flatten()
query_image_flatten = query_image_data.flatten()
```

Cosine Similarity

Image Similarity

- Compute Cosine Similarity

```python
def compute_cosine_similarity(A, B):
    cosine = np.dot(A,B)/(np.linalg.norm(A)*np.linalg.norm(B))
    return cosine
```

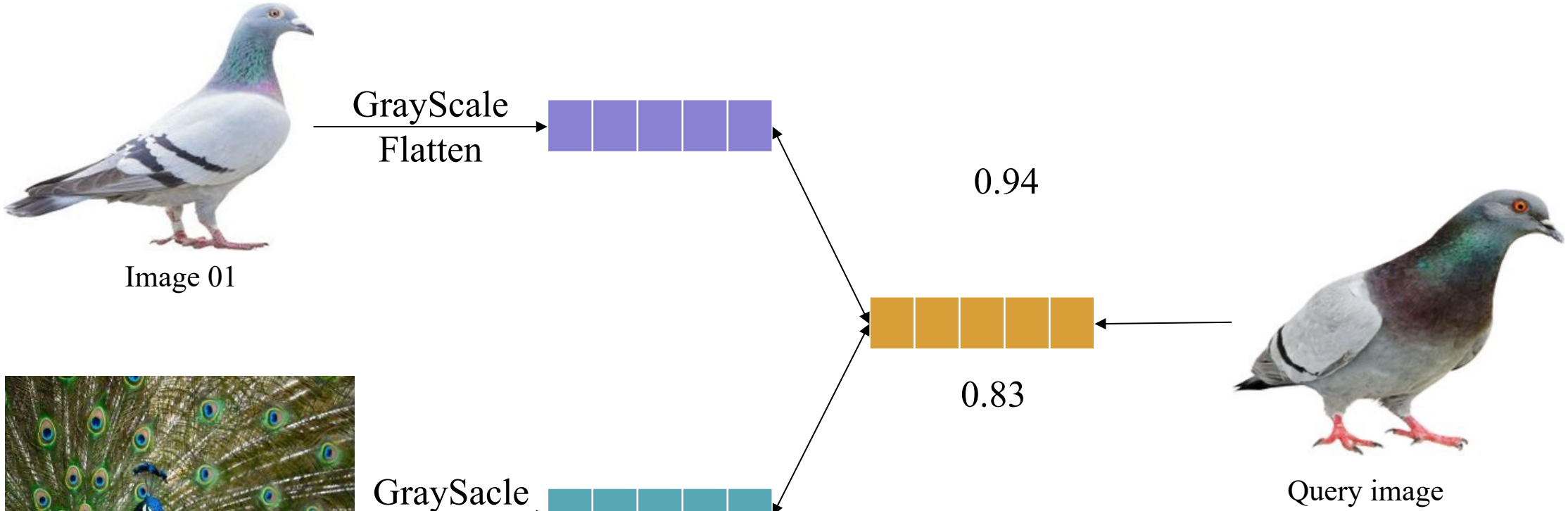
```python
cosine1 = compute_cosine_similarity(
    image1_flatten, query_image_flatten
)
cosine2 = compute_cosine_similarity(
    image2_flatten, query_image_flatten
)
cosine1, cosine2
```

✓ 0.0s

```
(np.float64(0.9388725624612365), np.float64(0.8301642549068863))
```

Cosine Similarity

Result



Summary

Vector Operations

- Addition
- Subtraction
- Multiplication
- Division
- Dot Product

Cosine Similarity

- Compute Cosine Similarity
- Image Similarity

$$\text{cs}(\vec{x}, \vec{y}) = \frac{\vec{x}.\vec{y}}{\|\vec{x}\|\|\vec{y}\|} = \frac{\sum_1^n x_i y_i}{\sqrt{\sum_1^n x_i^2}\sqrt{\sum_1^n y_i^2}}$$

Matrix Operations

- Addition
- Subtraction
- Multiplication
- Division
- Dot Product
  (Matrix-vector & Matrix-Matrix)
- Transpose
- Determinant
- Inverse

Thanks!

Any questions?